DEPARTMENT OF SOFTWARE ENGINEERING
Faculty of Computing & Information Technology (FCIT)
University of the Punjab, Lahore – PAKISTAN



MedBill D1 Agile Report

Claim Intake Validation and CMS Preview

BSEF23-51 MedBill

Bachelors of Science in Software Engineering (2023-2027)

By

Hasan Abdullah BSEF23M552

Irfan Saleem BSEF23M553

Supervised by:

Mr Umair Babar
Primary Supervisor

FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY, UNIVERSITY OF THE PUNJAB, LAHORE.



Contents

1. Product Overview....2
2. System Design High Level....2
3. Features and Scope....4
4. Product Backlog....5
5. Prototype Demonstration....6
6. Progress Snapshot....7
7. Definition of Done....7
8. Development Approach....8
9. CI CD Plan Brief....8
10. Next Steps Semester 8....9

1. Product Overview

MedBill is a medical billing project for small clinics and billing teams. Our D1 increment implements the first usable claim-intake slice: a biller signs in, creates a claim manually or from a PDF, reviews the patient, insurance and service fields, runs explainable checks, saves the validated draft and opens a CMS-1500-style preview.

Problem. Claim information arrives in different forms, and manual re-entry can miss patient identifiers, insurance details, dates, diagnosis codes, procedure codes and charges. D1 needs one inspectable path that turns this information into a structured draft without claiming that the whole billing lifecycle is complete.

Target users. A biller creates and reviews synthetic draft claims. An admin can also open the preserved technical PDF workspace. Signed sessions and role checks protect the claim and document APIs. The public demonstration must still use synthetic data only.

D1 solution. Both input paths lead to the same editable claim, deterministic validation result, PostgreSQL record and printable CMS-1500-style preview. A validated draft means the required D1 format and presence checks passed; it does not mean a payer accepted the claim.

2. System Design High Level

One Next.js application serves the React interface and Node.js Route Handlers on Vercel. Signed demo sessions enforce Biller and Admin roles. Prisma stores patients, insurance records, claims and source-document metadata in Supabase PostgreSQL. Cloudflare R2 stores private PDFs. PDF.js reads embedded text and Tesseract.js handles scanned pages before the biller reviews every extracted value.

Next.js browser UI	Next.js Node.js API	Persistent services
Sign in, manual/PDF entry, review, validation, history, CMS preview	Signed sessions, extraction, validation, claim persistence, protected technical APIs	Prisma/PostgreSQL for records; private R2 for source PDFs

Technology choices

- Frontend: Next.js App Router, React and TypeScript with one guided claim workflow and a printable CMS-1500-style preview.
- Backend: Next.js Node.js Route Handlers for authentication, claim listing and creation, PDF extraction and the admin technical workspace.
- Database: Supabase PostgreSQL through Prisma ORM, with Patient, Insurance, Claim and Document entities.
- Files and extraction: private Cloudflare R2 storage; PDF.js embedded-text extraction and bundled Tesseract.js OCR. Extracted values always require human review.
- Deployment and checks: Vercel production, Prisma migrations, lint, TypeScript build and Playwright browser tests against local and production URLs.

3. Features and Scope

Core D1 capabilities

- Biller and Admin demo sign-in with an eight-hour signed session
- Manual patient, insurance and single-service claim entry
- PDF upload with embedded-text extraction or local OCR
- Human review and correction before validation
- Explainable required-field, code-format, date and amount checks
- PostgreSQL claim persistence and recent-claim history
- CMS-1500-style preview with browser print or Save as PDF

D1 implementation boundary

Implemented	Partial or limited	Remaining proposal work
Biller and Admin demo roles	Demo accounts, not production identity management	Organization users, password recovery and audit administration
Manual patient and insurance entry	Creates a new patient and policy per claim	Full patient search, edit, deduplication and eligibility
PDF extraction and human review	Label-based extraction; OCR can misread fields	Broader document formats and measured extraction accuracy
Claim validation and persistence	Presence and format checks only	Official code lookup, payer rules and electronic submission
CMS-1500-style printable preview	Not an official clearinghouse transaction	Certified export/submission and acknowledgement handling
Recent claims and technical evidence	No payment or denial lifecycle	Payments, denials, financial reports and measured ML

This increment does not submit claims to payers, verify eligibility, post payments, manage denials, reconcile finances or claim that a trained denial-risk model exists. The older AI document router remains available only to the Admin as technical evidence and is not the supervisor-facing D1 product flow.

4. Product Backlog

The Must stories below define the demonstrated claim-intake increment. Done refers only to the stated acceptance check. Later billing, integration and measured machine-learning work remains in the backlog.

ID	Priority	User story and acceptance check	D1 status
AUTH-01	Must	As a biller or admin, I sign in and protected pages/APIs refuse unauthenticated access.	Done
PAT-01	Must	As a biller, I enter the minimum patient and insurance fields needed for a draft claim.	Done
CLM-01	Must	As a biller, I enter provider, service, diagnosis, procedure and charge information.	Done
PDF-01	Must	As a biller, I upload a valid PDF and receive editable extracted fields; invalid files are refused.	Done
REV-01	Must	As a biller, I review and correct all manual or extracted values before validation.	Done
VAL-01	Must	As a biller, I see explainable checks and cannot save while required checks fail.	Done
DB-01	Must	As a biller, I save a validated draft and find it in recent claim history.	Done
CMS-01	Must	As a biller, I open a CMS-1500-style preview and print or save it as PDF.	Done
TECH-01	Should	As an admin, I can open the older technical PDF workspace without exposing it to billers.	Done
PAY-01	Later	As a biller, I post payments, adjustments and denial outcomes.	Planned

5. Prototype Demonstration

Open https://medbill-steel.vercel.app and sign in with the Biller demo account. First show manual entry: use synthetic patient and insurance data, enter one service line, review it, validate it and open the CMS-1500-style preview. Then upload the supplied synthetic claim PDF, confirm that the same editable form is populated, correct any extraction error and complete the same validation and preview path. Use the Admin role only if the supervisor asks to see the preserved technical workspace.

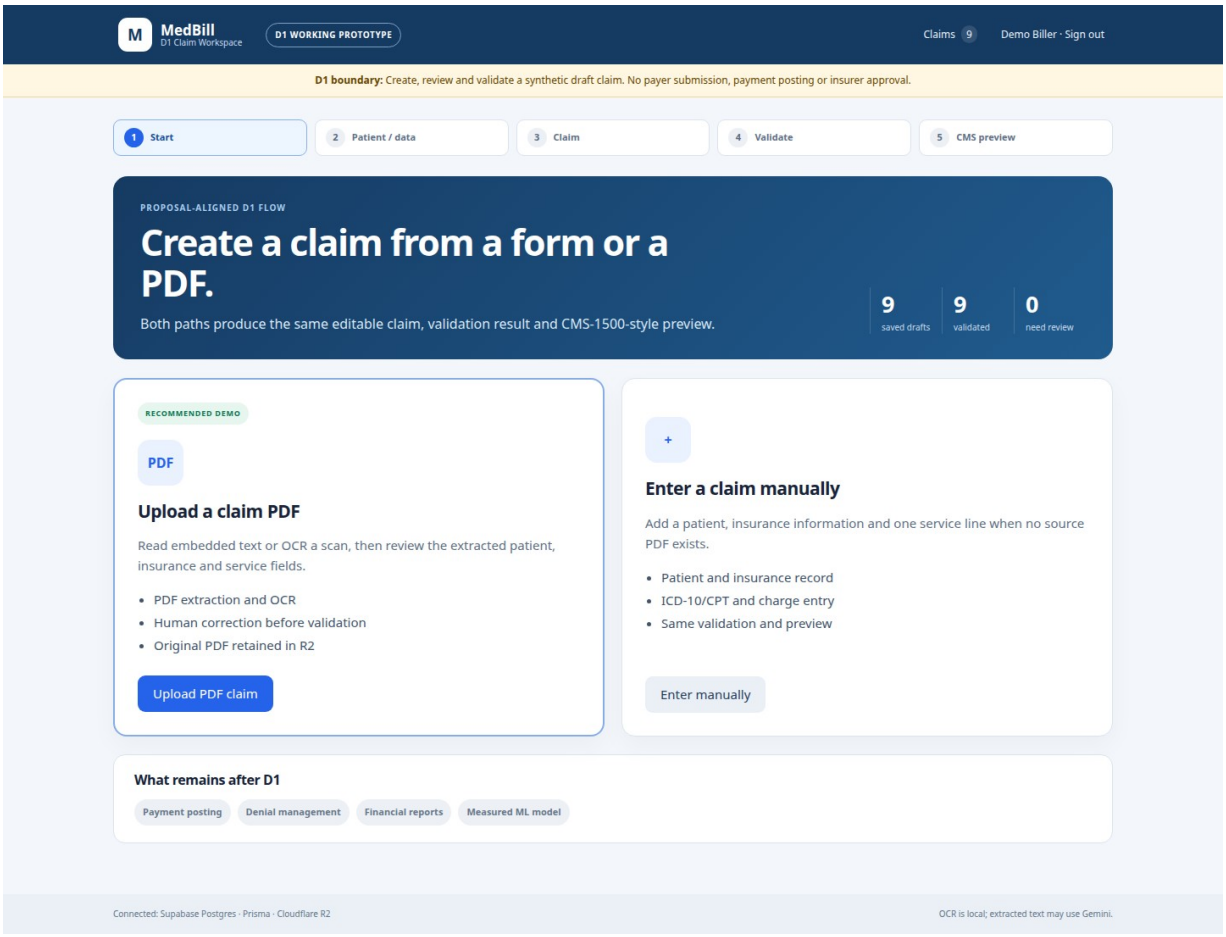


Figure 1 Deployed D1 claim workspace after production verification

Current limitations

- The public deployment is for synthetic data only. It has a 4 MB PDF upload limit and an eight-hour demo session.
- D1 checks validate required fields and recognizable formats. They do not verify insurer eligibility, medical necessity, official code-set status or reimbursement.
- The CMS-1500-style screen is a printable preview, not an official electronic claim submission. Payment posting, denials, financial reports and a measured denial-risk model remain later work.

6. Progress Snapshot

Live app: https://medbill-steel.vercel.app. GitHub: https://github.com/hasanabdullah919/medbill_bsef23-51. D1 runtime source: commit de3e418. Vercel production deployment: dpl_JD4EnQXRBKWBn3kcnQY5UvmnUsMa. Production verification was completed on 14 October 2026.

Evidence	Result	Meaning
Vercel health endpoint	Passed	Production PostgreSQL and R2 are reachable
Local lint and production build	Passed	ESLint, TypeScript and Next.js compilation succeeded
Local Playwright suite	5 of 5 passed	Authentication, manual flow, PDF flow, invalid login and admin access
Production Playwright suite	5 of 5 passed	The same complete behavior passed on the live URL
Prisma migration	Applied	Patient, Insurance and Claim tables exist in production PostgreSQL
Unauthenticated document API	HTTP 401	Older PDF data is no longer publicly accessible

The evidence folder includes the live D1 workspace screenshot and the production test record. Synthetic PDFs support the extraction demonstration. The repository contains the active source, Prisma migration and browser tests without credentials, dependencies or generated build output.

7. Definition of Done

A story is Done only when its stated D1 behavior passes with synthetic data. This does not mean payer integration or the wider product is complete.

- Protected pages and APIs require a valid role-checked session.
- Manual and PDF inputs reach the same review, validation, persistence and preview path.
- Lint, build, migration, local tests and production tests pass.
- The interface and report state the D1 boundary and use no private data or secrets.

8. Development Approach

The proposal uses two-week Scrum sprints with planning, review and retrospective. This increment delivers one vertical slice from claim input to a stored, printable draft. Later sprints should extend that path with payer integration, payment and denial workflows, reporting and measured machine-learning evidence.

Branching strategy

- main contains the published D1 application and aligned submission documentation.
- Feature and fix branches remain the proposed path for isolated sprint work.
- Pull-request review, protected checks and a develop integration branch remain planned team controls.

Change workflow

Planned workflow: refine a story, implement a vertical slice, run lint/build/tests, obtain peer review, demonstrate the deployed behavior and update the report. Current production deployment uses Vercel CLI; automatic GitHub deployment and hosted CI are not enabled.

Team responsibilities

- Hasan Abdullah: proposed Scrum Master and backend/AI lead, as listed in the proposal.
- Irfan Saleem: proposed Product Owner and frontend/database lead, as listed in the proposal.
- Both students should be able to explain setup, rules, limitations, tests and the live demonstration.

9. CI CD Plan Brief

Current checks run before manual deployment. Hosted GitHub Actions and protected-branch gates remain planned. Vercel builds the Next.js application, while Playwright verifies the complete user flow against the deployed URL.

Stage	Check	Current state or planned gate
Local code	npm run lint; npm run build; npm test	Executed before deployment
Database	Prisma generation and migrations	Applied to production PostgreSQL
Live system	Playwright against production plus health/API checks	Executed after deployment
Hosted CI	GitHub Actions, peer review and protected branches	Planned

10. Next Steps Semester 8

Target	Planned outcome
Scope review	Confirm the D1 claim-intake, validation and preview slice against the proposal.
Sprint 1	Add full patient/policy search, editing, deduplication and audit events.
Sprint 2	Confirm official CMS-1500 export and payer submission requirements; add code and eligibility services.
Sprint 3	Implement payment, adjustment, denial and outstanding-balance workflows with fixtures.
Sprint 4	Build reconciled reports and an approved, measured denial-risk experiment.
Release	Add CI/review gates, operational identity controls, monitoring and privacy review.

Demonstrate the complete D1 claim-intake slice and request supervisor confirmation that this is the correct first increment. The next work should add official claim export or submission requirements, eligibility and code lookup, payments and denials, reconciled financial reports, and a properly evaluated denial-risk model.